OKX DEX Router EVM

Incremental Security Audit Report

Prepared by

OKX Web3 Audit Team

Report compilation date

05 November 2026

Contents

1 Overview **2**

1.1 Project Introduction 2

1.2 Audit Summary 2

1.3 Audit Scope 2

1.4 Revision History 2

2 Audit Summary **3**

2.1 Audit Methodology 3

2.2 Audit Process 3

2.3 Risk Classification and Description 4

2.4 Vulnerability Checklist 4

3 Vulnerabilities **5**

3.1 Medium - Missing Output Amount Validation for Exact-Output Swaps 5

3.2 Medium - Native Adapter Refunds Remaining Input to the Router Caller 6

3.3 Low - Returned Input Consumption Excludes Source-Token Commission 7

3.4 Low - Unchecked Token Transfer Return Value in Direct Commission Transfers 8

3.5 Low - Excess Permit2 Input Is Silently Swapped on the Direct-to-Pool Path 9

3.6 Low - Unexpected Native Value Can Remain Locked During WETH Unwrapping 10

3.7 Informational - Incomplete Coverage of Fee-on-Transfer Token Support 11

4 Disclaimer **12**

5 About OKX Web3 Audit Team **13**

1. Overview

1.1 Project Introduction

OKX DEX Router is a decentralized exchange aggregator that integrates multiple protocols to provide token swap routes on EVM chains. This incremental security audit report consolidates three source reviews: v1.0.8 ERC-8021 compatibility, v1.0.9 DexRouter refactoring, and v1.0.10 RFQ anti-toxic-flow protection.

This incremental report covers ERC-8021 compatibility, the DexRouter refactoring and RFQ anti-toxic-flow protection as a single review record. The report compilation date is 05 November 2026; it is not a claimed audit completion date. Findings and dispositions are consolidated from the source reports, rather than a new independent code audit.

1.2 Audit Summary

Ecosystem	EVM	**Language**	Solidity
Repository			
Base Commit			
Final Commit			

1.3 Audit Scope

The audit covers the Router entry point, router implementations and related dependencies, the ERC-8021 changes in DexRouter and CommissionLib, and the RFQ caller-authentication changes in the DexRouter repository. The reviewed Adapter files are PMMAdapter.sol and NativePmmAntiToxicAdapter.sol.

```
GitHub-Web3-DEX-EVM/
  contracts/8/
    DexRouter.sol
    router/                    (all router implementations)
    libraries/
      CommissionLib.sol        (ERC-8021 compatibility)
      CallerAuth.sol           (RFQ authentication)
      CommonLib.sol
      Constants.sol
    adapter/
      PMMAdapter.sol
      NativePmmAntiToxicAdapter.sol
    Related router dependencies
```

The refactoring also describes separating exact-output contracts and dependencies into `contracts/exactout/`. The requested follow-up review after that split is not independently verified in this report.

1.4 Revision History

Version	**Description**
v1.0.8	ERC-8021 compatibility
v1.0.9	DexRouter refactoring
v1.0.10	RFQ anti-toxic-flow protection

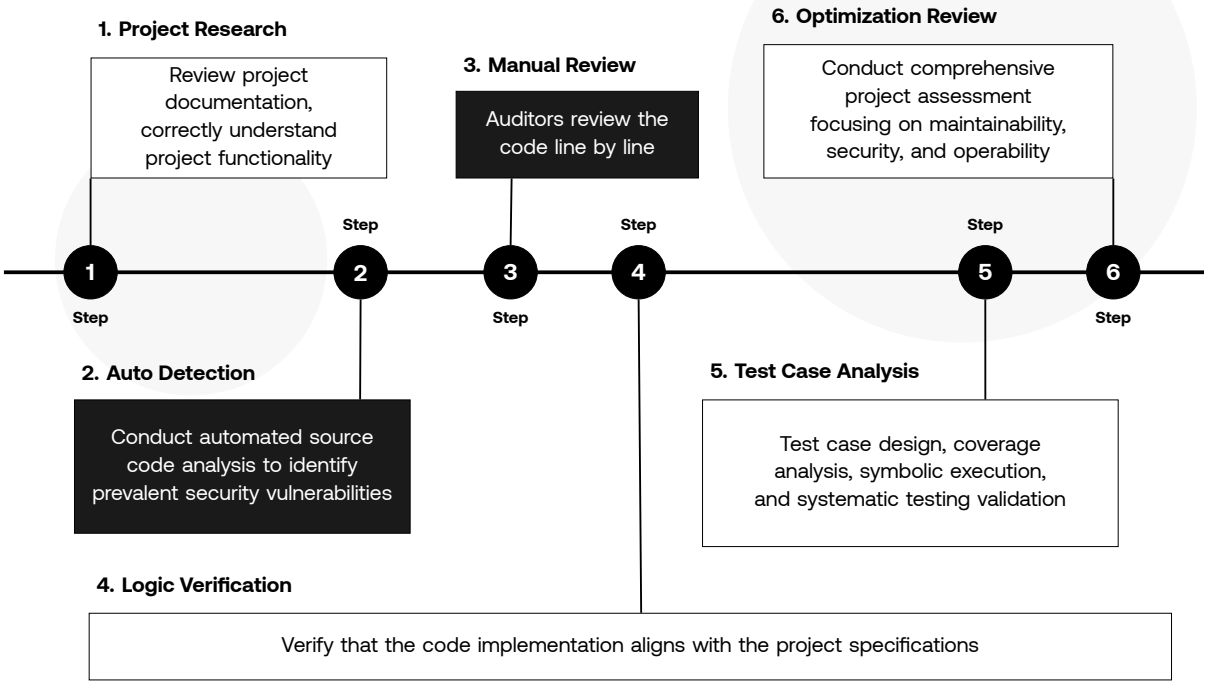
2. Audit Summary

2.1 Audit Methodology

Security review considers the project design, operating principles, implementation and function call relationships. Reported issues are documented with their impact, relevant code, recommendations and the project response. This section outlines the general audit methodology; it is not a claim that new testing was performed for this compilation.

2.2 Audit Process

The standard smart contract security audit workflow covers a 6-phase process: Project Research, Automated Detection, Manual Review, Logic Verification, Test Case Analysis, and Optimization Review. During manual auditing, auditors perform comprehensive code review to identify vulnerabilities and provide detailed solutions. Findings are communicated to the project team for response and follow-up. The diagram summarizes that general workflow; the version-specific source reports govern the scope and conclusions reproduced here.



2.3 Risk Classification and Description

Risk items are classified into 5 levels: Critical, High, Medium, Low, and Informational. Critical risks require immediate resolution and re-audit before final report delivery; unresolved critical risks result in audit failure. High risks must be addressed but are less urgent; failure to resolve also results in audit failure. Medium risks indicate potential exposure and require clear documentation of project team notification and response status without affecting report delivery. Low risks and informational items involve compliance or code detail issues that may be deferred without impacting report delivery.

Risk Level	Icon	Risk Description
Critical		Fatal risks requiring immediate resolution
High	!!	High-risk vulnerabilities that will cause similar issues, must be resolved
Medium		Medium-risk vulnerabilities with potential impact, should be resolved
Low	!	Low-risk issues with improper handling or warning triggers, can be deferred
Informational	!	Optimization opportunities, deferrable but recommended for resolution

2.4 Vulnerability Checklist

The vulnerability checklist is divided into two parts: one part is the vulnerability summary of the project audit, and the other part is the detailed vulnerability list.

Vulnerability Summary

Critical	High	Medium	Low	Informational	Total
0	0	2	4	1	7

Vulnerability List

No.	Severity	Vulnerability	Category	Status
1	Medium	Missing Output Amount Validation for Exact-Output Swaps	Output validation	Fixed
2	Medium	Native Adapter Refunds Remaining Input to the Router Caller	Refund logic	Fixed
3	Low	Returned Input Consumption Excludes Source-Token Commission	Accounting logic	Fixed
4	Low	Unchecked Token Transfer Return Value in Direct Commission Transfers	Return-value check	Fixed
5	Low	Excess Permit2 Input Is Silently Swapped on the Direct-to-Pool Path	Input validation	Fixed
6	Low	Unexpected Native Value Can Remain Locked During WETH Unwrapping	Input validation	Fixed
7	Informational	Incomplete Coverage of Fee-on-Transfer Token Support	Token compatibility	Confirmed

Status Definitions

- **Fixed**: The source records a remediation and the auditor confirms the response. Any stated configuration assumptions remain applicable.
- **Confirmed**: The auditor acknowledges the project response or accepts the behavior by design; a universal code fix is not implied.
- **Open**: No confirmed remediation or accepted response is recorded.

3. Vulnerabilities

The seven findings are ordered by severity: two Medium, four Low and one Informational. Each finding records its code location, recommendation and project response.

3.1 Medium - Missing Output Amount Validation for Exact-Output Swaps

Source: v1.0.9 / No. 1

Location	File	Category	Status	Severity
Lines 176-219	`UnxswapV3ExactOutRouter.sol`	Output validation	Fixed	⚠ Medium

Description

After _executeSwapRecursive completes, _uniswapV3SwapExactOut checks only that consumedAmount does not exceed maxConsume. It does not verify that the receiver obtained the requested amountOut on the non-WETH output path. A Uniswap V3 pool can partially fill an exact-output request when available liquidity is insufficient and return without reverting. The input limit still passes because less input is consumed, while the user receives fewer output tokens than requested.

The WETH unwrapping path has an additional balance constraint: _unwrapWETH(amountOut, receiver) reverts if the required WETH is unavailable. The reported omission concerns the output path without WETH unwrapping.

Related Code

Excerpt from the submitted version; this is not the fixed implementation. Ellipses mark omitted code. Snippet line numbers are local to this excerpt; the source location is listed above.

```
1   IDexRouter.ExtraData memory extraData_;
2   extraData_.commissionInfo = extraData.commissionInfo;
3   extraData_.permit2Info = extraData.permit2Info;
4   _executeSwapRecursive(
5       payer,
6       middleReceiver,
7       amountOut,
8       mode,
9       pools,
10      extraData_
11  );
12  // If WETH needs to be unwrapped, unwrap and send ETH to the actual receiver.
13  if (unwrapWeth) {
14      _unwrapWETH(amountOut, receiver);
15  }
16  // Get the actual amount of input tokens consumed and reset the state variable to save gas.
17  uint256 consumedAmount = TempStorage.getAmountConsumed();
18  TempStorage.setAmountConsumed(0);
19  // Ensure the consumed amount does not exceed the maximum allowed.
20  require(consumedAmount <= maxConsume, "ExcessiveInputAmount");
```

Recommendation

Capture the actual output amount returned by the final pool swap and require it to be at least amountOut.

Project Team Feedback

Accepted. A check that the return value of LastUniV3Pool.swap is greater than or equal to amountOut has been added.

Re-audit Result

Confirmed by the auditor in the source report.

3.2 Medium - Native Adapter Refunds Remaining Input to the Router Caller

Source: v1.0.10 / No. 1

Location	File	Category	Status	Severity
Line 71	NativePmmAntiToxicAdapter.sol	Refund logic	Fixed	⚠ Medium

Description

NativePmmAntiToxicAdapter reads the caller marker at offset -64 and uses dexRouterCaller as the refund receiver, without using the ORIGIN_PAYER/refundTo marker at offset -32. The PMM refund handler instead reads offset -32.

For dagSwapByOrderId or dagSwapTo in INVEST mode, when extraData.refundTo differs from msg.sender, surplus sellerToken can be sent to msg.sender instead of the intended refund address.

Related Code

Excerpt from the submitted version; this is not the fixed implementation. Ellipses mark omitted code. Snippet line numbers are local to this excerpt; the source location is listed above.

```
1   function _nativePmmSwap(address to, bytes memory moreInfo) internal {
2       MoreInfo memory info = abi.decode(moreInfo, (MoreInfo));
3       INativeRfqPoolV3.RFQTQuote memory quote = info.quote;
4       require(quote.recipient == address(this), "bad recipient");
5       ...
6   address dexRouterCaller = _extractDexRouterCaller();
7       require(
8           dexRouterCaller != address(0) && info.beneficiary == dexRouterCaller,
9           "beneficiary mismatch"
10      );
11      _executeTrade(to, quote, dexRouterCaller);
12  }
13  function _executeTrade(
14      address to,
15      INativeRfqPoolV3.RFQTQuote memory quote,
16      address refundReceiver
17  ) internal {
18      (uint256 value, uint256 actualAmountIn) = _prepareInput(quote);
19      ...
20      _transferAssetBalance(outputToken, to);
21      _transferAssetBalance(quote.sellerToken, refundReceiver);
22  }
```

Recommendation

Read the refund recipient from the ORIGIN_PAYER marker at offset -32, consistently with the PMM refund handler.

Project Team Feedback

The issue has been fixed.

Re-audit Result

Confirmed by the auditor.

3.3 Low - Returned Input Consumption Excludes Source-Token Commission

Source: v1.0.9 / No. 2

Location	File	Category	Status	Severity
Line 59	`ExactOutSwapHelper.sol`	Accounting logic	Fixed	! Low

Description

UnxswapExactOutRouter and UnxswapV3ExactOutRouter pass consumeAmount to ExactOutSwapHelper.afterSwap through a newly constructed AfterSwapParams structure. Adding fromTokenCommissionAmount updates only the field in that structure. It does not update the caller's local consumeAmount variable.

The outer function therefore returns the swap input amount without the source-token commission, contrary to its documented meaning of total source tokens consumed including commission. Frontend or SDK accounting based on this result can understate the user's total expenditure.

Related Code

Excerpt from the submitted version; this is not the fixed implementation. Ellipses mark omitted code. Snippet line numbers are local to this excerpt; the source location is listed above.

```
1   function afterSwap(IDexRouter.AfterSwapParams memory afterSwapParams) internal {
2           // Handle commission from the source token if applicable.
3           if (
4               afterSwapParams.commissionInfo.isFromTokenCommission &&
5               afterSwapParams.commissionInfo.commissionRate > 0
6           ) {
7               CommissionLib.doCommissionFromToken(
8                   afterSwapParams.commissionInfo,
9                   IDexRouter.SwapCache({
10                      payer: afterSwapParams.payer,
11                      refundTo: address(0),
12                      receiver: afterSwapParams.receiver,
13                      toToken: afterSwapParams.toToken
14                  }),
15                  afterSwapParams.fromTokenCommissionAmount,
16                  false
17              );
18              // Add the commission to the total consumed amount.
19              afterSwapParams.consumeAmount += afterSwapParams.fromTokenCommissionAmount;
20          }
21          ...
22      }
23  }
```

Recommendation

Return the updated consumeAmount explicitly from afterSwap and assign the return value in each caller.

Project Team Feedback

Accepted. afterSwap now returns the updated value, and the calling functions update their local consumeAmount variables.

Re-audit Result

Confirmed by the auditor in the source report.

3.4 Low - Unchecked Token Transfer Return Value in Direct Commission Transfers

Source: v1.0.9 / No. 3

Location	File	Category	Status	Severity
Lines 176-200	`CommissionLib.sol`	Return-value check	Fixed	! Low

Description

_transferTokenDirect invokes transferFrom in assembly and checks only whether the low-level call reverts. It does not validate the returned data. For a token that signals failure by returning false, the low-level call can succeed while the transfer fails.

Execution can continue and emit CommissionFromToken even though the referrer did not receive the commission. This creates a discrepancy between the emitted event and the actual token movement.

Related Code

Excerpt from the submitted version; this is not the fixed implementation. Ellipses mark omitted code. Snippet line numbers are local to this excerpt; the source location is listed above.

```
1   function _transferTokenDirect(token, from, to, amount) {
2       let freePtr := mload(0x40)
3       mstore(0x40, add(freePtr, 0x64)) // calldata size == 0x64
4       mstore(
5           freePtr,
6           0x23b872dd00000000000000000000000000000000000000000000000000000000
7       ) // transferFrom(address,address,uint256)
8       mstore(add(freePtr, 0x04), from)
9       mstore(add(freePtr, 0x24), to)
10      mstore(add(freePtr, 0x44), amount)
11      let success := call(
12          gas(),
13          token,
14          0,
15          freePtr,
16          0x64,
17          0,
18          0x20
19      )
20      if eq(success, 0) {
21          _revertWithReason(
22              0x0000001b207472616e7366657220746f6b656e206469726563742066616696c00,
23              0x5f
24          ) // "transfer token direct fail"
25      }
26  }
```

Recommendation

Validate both low-level call success and the token return data, following the safe-transfer handling already used by _sendToken in the same file.

Project Team Feedback

Accepted. Return-value checking in _transferTokenDirect has been fixed.

Re-audit Result

Confirmed by the auditor in the source report.

3.5 Low - Excess Permit2 Input Is Silently Swapped on the Direct-to-Pool Path

Source: v1.0.9 / No. 4

Location	File	Category	Status	Severity
Line 196	SmartSwapRouter.sol	Input validation	Fixed	! Low

Description

In PERMIT2_SIGNATURE mode with a single batch and a single assetTo destination, permitTransferFrom transfers the full fromTokenAmount directly to the destination pool. The payer is then set to address(0), so downstream execution does not transfer tokens again.

_smartSwapInternal checks only totalBatchAmount <= fromTokenAmount. If totalBatchAmount is smaller, the excess is already at the pool and cannot be returned by the normal refund path. Adapters that derive their input from the pool or adapter balance can silently swap the excess as well, exceeding the amount expressed by batchesAmount.

Related Code

Excerpt from the submitted version; this is not the fixed implementation. Ellipses mark omitted code. Snippet line numbers are local to this excerpt; the source location is listed above.

```
1   address onlyOneAssetTo = (batches.length == 1 && batches[0][0].assetTo.length == 1)
2       ? batches[0][0].assetTo[0]
3       : address(0);
4
5   if (onlyOneAssetTo == address(0) || ...) {
6       to = address(this);
7   } else {
8       to = onlyOneAssetTo;
9   }
10  transferDetails[0] = ISignatureTransfer.SignatureTransferDetails({
11      to: to,
12      requestedAmount: inputAmount + fromTokenCommissionAmount
13  });
14
15  } else if (mode == _MODE_PERMIT2_SIGNATURE) {
16      require(payer == address(0), "permitTransferFrom must be called before");
17  }
18
19  require(totalBatchAmount ⩽ baseRequest.fromTokenAmount, ...);
```

Recommendation

Require totalBatchAmount == fromTokenAmount whenever payer is address(0) because Permit2 has already transferred the funds directly to the pool.

Project Team Feedback

Accepted. A branch now requires totalBatchAmount == fromTokenAmount when payer is address(0).

Re-audit Result

Confirmed by the auditor in the source report.

3.6 Low - Unexpected Native Value Can Remain Locked During WETH Unwrapping

Source: v1.0.9 / No. 5

Location	File	Category	Status	Severity
Line 119	SwapWrapRouter.sol	Input validation	Fixed	! Low

Description

The swapWrap and swapWrapToWithBaseRequest entry points are payable. When reversed is true, SwapWrapRouter transfers WETH in and unwraps it, but does not require msg.value to be zero.

The source-token refund branch runs only when reversed is false. Native currency supplied accidentally with an unwrapping request can therefore remain in the contract.

Related Code

Excerpt from the submitted version; this is not the fixed implementation. Ellipses mark omitted code. Snippet line numbers are local to this excerpt; the source location is listed above.

```
1   if (reversed) {
2               TransferLib.transferInternal(
3                   swapCache.payer,
4                   address(this),
5                   mode | uint256(uint160(_WETH)),
6                   amount
7               );
8               IWETH(_WETH).withdraw(amount);
9               if (middleReceiver != address(this)) {
10                  // Invariant: `middleReceiver != address(this)` implies no toToken commission and no trim
11                  (bool success, ) = payable(middleReceiver).call{
12                      value: amount,
13                      gas: NATIVE_TOKEN_TRANSFER_GAS_LIMIT
14                  }("");
15                  require(success, "transfer native token failed");
16              }
17          } else {...}
18  ...
19  // Refund unused fromToken
20          if (!reversed) {
21              TransferLib.refundToken(_ETH, swapCache.refundTo);
22          }
23          ...
24  }
```

Recommendation

Require msg.value == 0 on the reversed branch.

Project Team Feedback

Accepted. A msg.value == 0 check has been added to the reversed branch.

Re-audit Result

Confirmed by the auditor in the source report.

3.7 Informational - Incomplete Coverage of Fee-on-Transfer Token Support

Source: v1.0.9 / No. 6

Location	File	Category	Status	Severity
Not specified	Not specified	Token compatibility	Confirmed	! Info

Description

Fee-on-transfer handling is implemented in selected paths, including balance-based input adjustment after Permit2 transfers and balance-delta handling in UnxswapRouter. The original observation asks whether equivalent handling is complete across other paths, especially the else branch of handlePermit2SigModeWithCommission.

The project team explains that inputAmount represents the amount that can be transferred from the payer's perspective. Its meaning differs between tokens staged in the router and tokens transferred directly downstream. Balance adjustment on the direct path would conflict with the totalBatchAmount equality check added for the Permit2 issue. Support is intentionally kept consistent with the existing production interfaces; exact-output and swapWrap paths do not support fee-on-transfer tokens.

Related Code

Excerpt from the submitted version; this is not the fixed implementation. Ellipses mark omitted code. Snippet line numbers are local to this excerpt; the source location is listed above.

```
1   if (to == address(this)) {
2       // Get balance of token because the token has been claimed to address(this).
3       uint256 balance = CommonLib.getBalanceOf(token, address(this));
4       // Calculate the inputAmount and fromTokenCommissionAmount according to the mode.
5       if (extraData.commissionInfo.isFromTokenCommission && toBCommission == TO_B_MODE) { // For
        ↪  Permit2Signature mode && ToB mode, the inputAmount and fromTokenCommissionAmount need to be
        ↪  calculated.
6           inputAmount = inputAmount * balance / (inputAmount + fromTokenCommissionAmount);
7           fromTokenCommissionAmount = balance - inputAmount;
8           return (address(this), inputAmount, fromTokenCommissionAmount);
9       } else { // For other situations (NoToB mode or no fromTokenCommission), the new inputAmount is the
        ↪  balance.
10          return (address(this), balance, fromTokenCommissionAmount);
11      }
12  } else {
13      return (address(0), inputAmount, fromTokenCommissionAmount);
14  }
```

Recommendation

Document the supported fee-on-transfer scenarios and preserve consistent amount semantics across transfer modes. If support is expanded, validate each affected swap, refund and commission path.

Project Team Feedback

The existing support boundaries and else-branch behavior are retained for compatibility. During this review, the team also identified an OrderRecord event inconsistency after balance adjustment. The stated remediation is to snapshot the original fromTokenAmount for that event on the affected Permit2 signature path. This snapshot is not added to exact-output or swapWrap paths because those paths do not support fee-on-transfer tokens.

Re-audit Result

The auditor confirmed the project response. The overall item is recorded as Confirmed because the support boundaries are accepted by design; it is not represented as universal fee-on-transfer support.

4. Disclaimer

This audit report only covers the specific audit types stated herein. We assume no responsibility for unknown security vulnerabilities outside this scope.

We rely on audit reports issued before existing attacks or vulnerabilities are published. For future or new vulnerabilities, we cannot guarantee project security impact and assume no responsibility.

Our security audit analysis should be based on documents provided by the project before report release (including contract code). These materials should not contain false information, tampering, deletion, or concealment. If provided materials are false, inaccurate, missing, tampered, deleted, concealed, or modified after report release, we assume no responsibility for resulting losses and adverse effects.

The project team should understand our audit report is based on provided materials and current technical capabilities. Due to institutional technical limitations, our report may not detect all risks. We encourage continued testing and auditing by the development team and stakeholders.

The project team must ensure compliance with applicable laws and regulations.

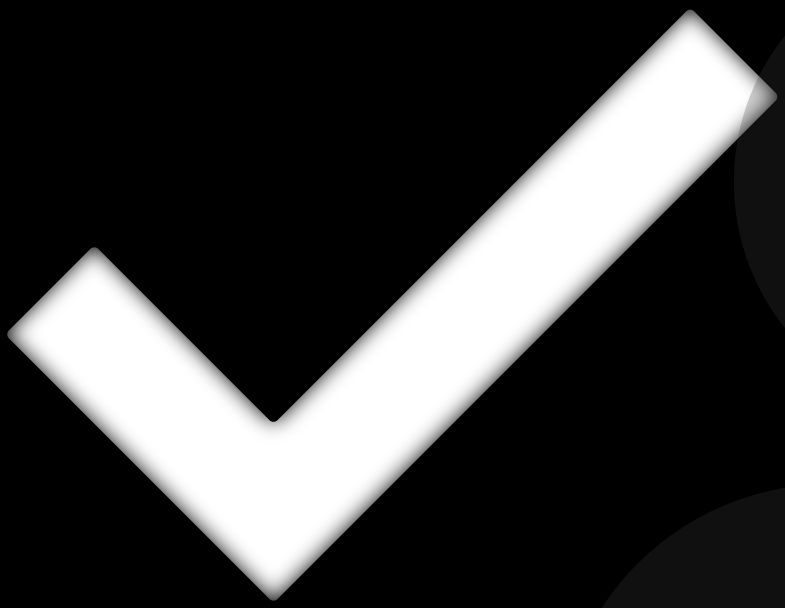
The audit report is for reference only. Its content, acquisition methods, usage, and related services cannot serve as basis for investment, taxation, legal, regulatory, or construction decisions. Without our prior written consent, the project team may not reference, cite, display, or distribute report content to third parties. Any resulting losses shall be borne by the project team.

This report does not cover contract compiler bugs or scope beyond programming languages. Smart contract risks from underlying vulnerabilities should be borne by the project team.

Force majeure includes unforeseeable, unavoidable events like wars, natural disasters, strikes, epidemics, and legal/regulatory changes preventing contract performance. When occurring, neither party breaches contract obligations. For unaffected economic responsibilities, the project team should pay for completed work.

5. About OKX Web3 Audit Team

OKX Web3 Audit Team specializes in blockchain security with expertise in smart contract auditing, token security assessment, and Web3 security tool development. We provide comprehensive security solutions for OKX's internal Web3 projects, conduct pre-listing token audits, and develop security tools to protect OKX Web3 wallet users. Our team combines automated analysis with manual review to deliver thorough security assessments and maintain the highest security standards in the Web3 ecosystem.



PASSED

This audit has been conducted to review the OKX DEX Router project's Solidity-based smart contracts running on EVM chains, examining design, architecture, and implementation to identify potential vulnerabilities

OKX Web3 Audit Team